A PROJECT REPORT ON

(BHARAT FINVEST INTERNAL OPERATIONS MANAGEMENT SYSTEM)

BY

(TEJAS SUDHAKAR KAMBLE)

M.Sc. (Computer Science) Part II, Sem IV

INDUSTRIAL TRAINING ORGANIZATION

Nityashree Infosystems Pvt. Ltd.

CLIENT DOMAIN ORGANIZATION

Bharat Finvest

Submitted To

SIR PARASHURAMBHAU COLLEGE, PUNE

(2025 – 2026)

COMPLETION CERTIFICATE

Completion Certificate

The company completion certificate from Nityashree Infosystems Pvt. Ltd. will be attached here after official issuance.

Shikshana Prasaraka Mandali's

SIR PARASHURAMBHAU COLLEGE

(EMPOWRED AUTONOMOUS)

TILAK ROAD, PUNE – 411 030.

Department of Computer Science

Certificate

This is to certify that the work in the entitled project of

_____

has been carried out by Mr./Mrs./Ms._____
as the partial fulfillment of the requirements of the M.Sc. (Computer Science) Part II, MSCOSDSC401 - Industrial training at

_____

for the last semester starting from _____ 20

Date : / /20

Head
Department of Science Computer

Teacher Incharge

1. *Internal Examiner*
2. *External Examiner*

DECLARATION

I, **Tejas Sudhakar Kamble**, hereby declare that this project report titled **Bharat Finvest Internal Operations Management System** is my original work carried out as part of the Industrial Training Project for M.Sc. (Computer Science) Part II, Sem IV at Sir Parashurambhau College, Pune.

This project was developed during industrial training at **Nityashree Infosystems Pvt. Ltd.** as a client-style internal operations management system for **Bharat Finvest**.

I confirm that:

- The work presented in this report has been implemented and validated in the repository `https://github.com/tejasskamble/bharat-finvest-ops-system`.
- The technical descriptions are based on the actual source code, database scripts, and DevOps files present in the project.
- Wherever external references are used, appropriate acknowledgements are provided.

Place: Pune
Date: _____

Signature: _____
Name: Tejas Sudhakar Kamble

ACKNOWLEDGEMENT

I express my sincere gratitude to Sir Parashurambhau College, Pune, Department of Computer Science, for providing the academic platform to complete this Industrial Training Project.

I am thankful to **Nityashree Infosystems Pvt. Ltd.**, my training organization, for mentorship, professional guidance, and practical exposure during this project period.

This project was developed during industrial training at Nityashree Infosystems Pvt. Ltd. as a client-style internal operations management system for Bharat Finvest.

I also acknowledge the operational domain context of **Bharat Finvest**, which helped shape realistic requirements for employee management, task tracking, attendance, and reporting workflows.

I would like to thank my faculty mentors and evaluators for their continuous guidance throughout requirement analysis, system design, implementation, QA, and documentation stages.

I also acknowledge the support received from peers and colleagues during project discussions, testing reviews, and final quality checks.

Finally, I thank my family for their encouragement and support during the completion of this project.

ABSTRACT

This report presents the design and implementation of the **Bharat Finvest Internal Operations Management System**, a full-stack web application developed to digitize core internal workflows such as employee management, task allocation, attendance marking, and operational reporting.

This project was developed during industrial training at **Nityashree Infosystems Pvt. Ltd.** as a client-style internal operations management system for **Bharat Finvest**.

The application uses a React + Vite frontend, an Express + Node.js backend, and MySQL as the relational data store. Authentication is implemented through JWT, passwords are secured using bcrypt hashes, and role-based access control is enforced for admin, manager, and employee roles.

The project also includes practical DevOps features such as Dockerized deployment, health checks, GitHub Actions continuous integration, and Windows-based backup/restore scripts. A one-click startup batch file is provided to simplify demonstrations.

The final system provides a reliable dashboard for operational visibility, structured CRUD flows for employees and tasks, attendance analytics, and PDF report export. The implementation reflects a realistic enterprise-style internal portal and is suitable for academic submission as well as live demonstration.

CONTENTS

1 Introduction to Problem **1**

2 Existing System **2**

3 Proposed System **3**

4 Scope of Work **4**

5 Feasibility Study **6**

5.1 Technical Feasibility 6

5.2 Operational Feasibility 6

5.3 Economic Feasibility 6

6 System Requirements **7**

6.1 Hardware Requirements 7

6.2 Software Requirements 7

7 Technology Stack **8**

8 System Architecture **9**

9 Database Design **10**

9.1 Tables 10

9.2 Relationship Notes 11

9.3 Seed Data Highlights 11

10 ER Diagram **12**

11 UML Diagrams **13**

11.1 Activity Diagram 14

11.2 Use Case Diagram 15

11.3 Class Diagram 16

11.4 Sequence Diagram 17

12 Module Description **18**
12.1 Login Module 18
12.2 Admin Dashboard Module 18
12.3 Employee Management Module 19
12.4 Task Management Module 19
12.5 Attendance Management Module 20
12.6 Reports Module 20
12.7 Profile, Branding, and Avatar Module 21
12.8 DevOps Module 21

13 User Interface Screens **22**
13.1 Login Screen 22
13.2 Dashboard Screen 23
13.3 Employees Screen 24
13.4 Tasks Screen 25
13.5 Attendance Screen 26
13.6 Reports Screen 27

14 API Design **28**
14.1 Authentication APIs 28
14.2 Dashboard API 28
14.3 Employee APIs 29
14.4 Task APIs 29
14.5 Attendance APIs 30
14.6 Health API 30
14.7 API Error Strategy 30

15 Security Implementation **31**
15.1 Authentication Security 31
15.2 Password Security with bcrypt 31
15.3 Role-Based Access Control 31
15.4 Database and Query Security 31
15.5 Client-Side Session Handling 32
15.6 Operational Security Hygiene 32

16 DevOps Implementation **33**
16.1 Dockerization Strategy 33
16.2 Docker Compose Orchestration 33
16.3 Health Checks 33
16.4 GitHub Actions CI Pipeline 34
16.5 Windows Automation Scripts 34
16.6 Environment-based Configuration 35

17 Testing Strategies **36**
17.1 Unit-level Validation 36
17.2 Integration Testing 36
17.3 System Testing 36
17.4 UI/UX Testing 37
17.5 Build and Runtime Verification 37
17.6 Regression Safety 37

18 Test Cases **38**

19 Limitations **40**

20 Future Scope **41**

21 Conclusion **42**

22 Bibliography & References **43**

23 Appendix **44**
23.1 A. Training and Client Context 44
23.2 B. GitHub Repository 44
23.3 C. Demo Credentials 44
23.4 D. Employee and Intern Seed Dataset 44
23.5 E. Essential Commands 45
23.6 F. Final QA Checklist Reference 46

INTRODUCTION TO PROBLEM

Financial operations teams in many organizations still depend on disconnected tools such as spreadsheets, WhatsApp updates, and manual registers for daily tracking. This creates fragmentation in information flow and delays operational decisions.

This project was developed during industrial training at **Nityashree Infosystems Pvt. Ltd.** as a client-style internal operations management system for **Bharat Finvest**.

In the Bharat Finvest client context, the following problem areas were identified:

- Employee master data and status were not centrally visible to all stakeholders.
- Task assignment and progress tracking lacked a single source of truth.
- Attendance updates required repetitive manual consolidation.
- Operational reports were time-consuming to prepare and difficult to standardize.
- Role-specific data access and accountability were difficult to enforce consistently.

The project addresses these problems by delivering a web-based internal portal that combines authentication, role-based workflow, operational dashboards, and structured reporting.

The structure and chapter sequence used in this report follow the academic format observed from the provided reference files (cover page style, certificate pattern, index pattern, and internal evaluation guideline sequence).

EXISTING SYSTEM

Before implementation, the typical process in similar setups was:

1. Employee details were maintained in spreadsheets.
2. Task updates were shared through informal communication.
3. Attendance was maintained through separate logs and periodic manual aggregation.
4. Reporting was generated manually, often by copying data between files.

Observed Limitations

- Data duplication and version conflicts.
- Limited traceability for who changed what and when.
- No unified role-based interface for admin/manager/employee users.
- Delayed reporting and inconsistent report formats.
- Difficult quality assurance due to multiple disconnected sources.

The absence of a centralized application directly impacted productivity and operational control.

PROPOSED SYSTEM

The proposed solution is a centralized internal operations management portal with secure login and role-aware access.

Core goals of the proposed system:

- Single application for employee, task, attendance, and reporting workflows.
- JWT-based login with backend authorization checks.
- Dashboard with live operational metrics.
- CRUD-based data management with validations and error handling.
- PDF report export for submission and review use cases.
- DevOps readiness through Docker, CI, health checks, and backup utilities.

User roles in system:

- **Admin**: Full access including employee deactivation and task deletion.
- **Manager**: Operational management access (employee/task/attendance updates except admin-only actions).
- **Employee**: Read-focused access to operational pages as configured by role restrictions.

SCOPE OF WORK

This project was developed during industrial training at **Nityashree Infosystems Pvt. Ltd.** as a client-style internal operations management system for **Bharat Finvest**.

The project scope covered end-to-end lifecycle stages:

1. Requirement Understanding

- Identify entities: users, employees, tasks, attendance.
- Define role behaviors and API-level restrictions.
- Define reporting requirements and demo credentials.

2. Application Development

- Frontend implementation with React, Vite, Bootstrap.
- Backend API implementation with Express and MySQL2.
- JWT authentication and authorization middleware.

3. Data Modeling

- Relational schema design for users, employees, tasks, attendance.
- Seed dataset creation including employee and intern records.

4. DevOps and Operations

- Docker Compose environment for frontend, backend, and MySQL.
- Health check API and container health validation.
- CI pipeline for backend import verification and frontend build.
- Backup and restore scripts for database operations.

5. QA and Stabilization

- Validation of all major UI actions and backend responses.
- Error handling, confirmation prompts, and loading states.

- Final data consistency verification for demo safety.

Out of Scope Clarification

- Official production deployment and live organization roll-out are outside the current academic project scope.
- No claim is made for official production adoption by Bharat Finvest in this report.

FEASIBILITY STUDY

5.1 Technical Feasibility

The required technology stack is mature and widely supported. React, Node.js, Express, and MySQL are stable for CRUD-centric enterprise applications. The project successfully integrates authentication, role middleware, charting, and PDF export.

5.2 Operational Feasibility

The target users (admin, manager, employee) interact through simple forms, tables, and filters. One-click launcher and explicit command documentation reduce operational friction during demos and daily usage.

5.3 Economic Feasibility

The project uses open-source frameworks and tooling. Local setup requires standard development machines and optional Docker Desktop. There is no mandatory paid dependency in the implementation.

SYSTEM REQUIREMENTS

6.1 Hardware Requirements

Component	Recommended Specification
Processor	Intel i5 / AMD Ryzen 5 or higher
RAM	Minimum 8 GB (16 GB preferred)
Storage	10 GB free space for code, packages, Docker images
Network	Internet required for dependency installation and Git operations

6.2 Software Requirements

Software	Purpose
Windows 10/11	Development and execution environment
Node.js (18+)	Runtime for backend and frontend tooling
npm	Package management
MySQL 8.0	Relational database
Git	Version control
Docker Desktop (optional)	Containerized deployment
VS Code	Source code editing

TECHNOLOGY STACK

Layer	Technology	Usage in Project
Frontend	React.js	Component-based UI for login, dashboard, employees, tasks, attendance, and reports pages.
Frontend Tooling	Vite	Fast development server and production build pipeline.
UI Framework	Bootstrap 5	Responsive layout, tables, modals, forms, alerts, buttons.
HTTP Client	Axios	API requests through centralized axios instance with auth header interceptor.
Charts	Chart.js + react-chartjs-2	Dashboard visualization for task status and attendance.
Reporting	jsPDF + jspdf-autotable	Client-side PDF exports for employee/task/attendance reports.
Backend Runtime	Node.js	JavaScript runtime for API server.
Backend Framework	Express.js	Route handling, middleware, JSON APIs, error responses.
Database Driver	mysql2/promise	Parameterized SQL query execution.
Database	MySQL 8.0	Persistent storage for users, employees, tasks, attendance.
Authentication	JWT	Stateless token authentication with Bearer headers.
Password Security	bcrypt	Secure password hash verification during login.
DevOps	Docker, Docker Compose	Multi-service orchestration: mysql, backend, frontend.
CI/CD	GitHub Actions	Automated dependency install, backend import check, frontend build.

SYSTEM ARCHITECTURE

The implemented system follows a three-tier architecture with an additional DevOps support layer.

- **Presentation Layer**: React + Vite frontend, served in development at port 5173 and through Nginx in Docker.
- **Application Layer**: Express backend at port 5000, exposing REST APIs under `/api/*`.
- **Data Layer**: MySQL database with normalized relational schema.
- **DevOps Layer**: Docker Compose orchestration, health checks, GitHub Actions CI, backup/restore scripts.

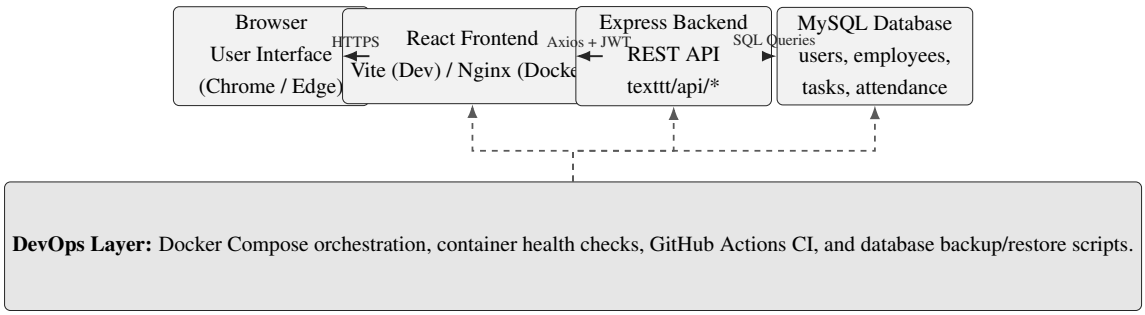

Figure 8.1: System Architecture Diagram

Request Flow Summary

1. User action in frontend triggers an Axios request.
2. Axios interceptor attaches JWT token (if present).
3. Backend middleware verifies token and role.
4. Controller executes parameterized SQL on MySQL.
5. JSON response updates UI state and renders feedback.

DATABASE DESIGN

The database schema is defined in `database/schema.sql` and seeded through `database/seed.sql`.

9.1 Tables

1. users

Stores authenticated application users.

- Primary Key: `id`
- Key fields: `name`, `email` (unique), `password_hash`, `role`, `department`
- Role values: `admin`, `manager`, `employee`

2. employees

Stores employee and intern profile records.

- Primary Key: `id`
- Foreign Key: `user_id` references `users(id)`
- Unique field: `employee_code`
- Key fields: `name`, `email`, `phone`, `department`, `designation`, `date_of_joining`, `status`
- Status values: `active`, `inactive`

3. tasks

Stores tasks assigned to employees.

- Primary Key: `id`
- Foreign Keys: `assigned_to` references `employees(id)`, `assigned_by` references `users(id)`

- Key fields: `title`, `description`, `priority`, `status`, `due_date`
- Priority values: `low`, `medium`, `high`
- Status values: `pending`, `in_progress`, `completed`

4. attendance

Stores attendance logs per employee and date.

- Primary Key: `id`
- Foreign Key: `employee_id` references `employees(id)`
- Unique composite key: `(employee_id, date)`
- Status values: `present`, `absent`, `half_day`, `leave`

9.2 Relationship Notes

- One user may be linked to zero or one employee profile (`users` to `employees`).
- One employee can have many tasks and many attendance records.
- One admin/manager user can assign many tasks.
- Cascade operations are handled carefully in APIs to avoid accidental data deletion in demo scenarios.

9.3 Seed Data Highlights

- Demo users: Admin (Akshay Shinde), Manager (Pravin Shinde), Employee (Ritesh Bhate).
- Employee table includes 8 regular employees and 4 interns (total 12 records).
- Seed includes realistic tasks and attendance records for reporting and dashboard visualization.

ER DIAGRAM

The following ER diagram represents the relational model used in the current project.

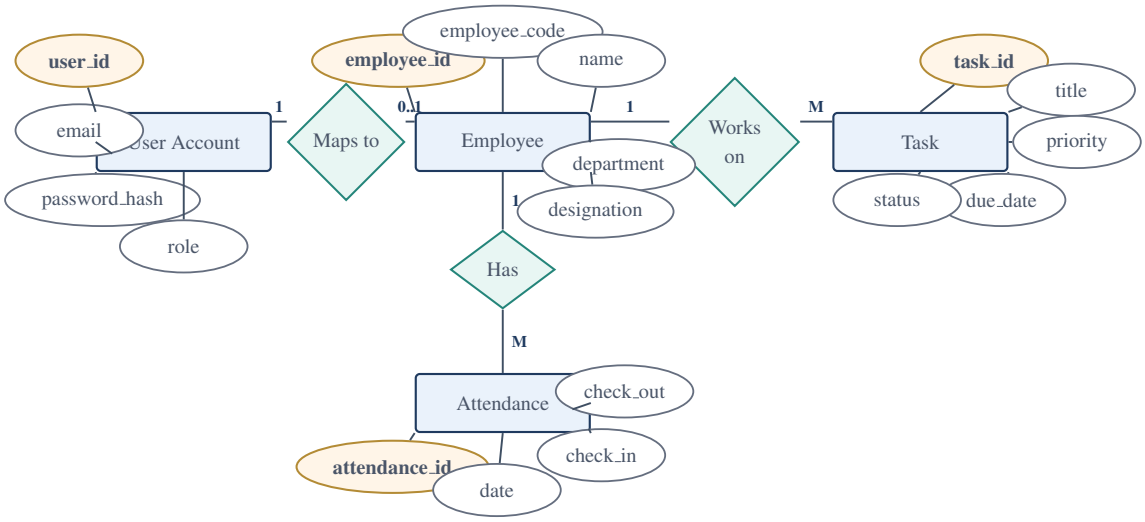

Figure 10.1: Entity Relationship Diagram

UML DIAGRAMS

The UML diagrams used to represent the system design are as follows:

- Activity Diagram
- Use Case Diagram
- Class Diagram
- Sequence Diagram

11.1 Activity Diagram

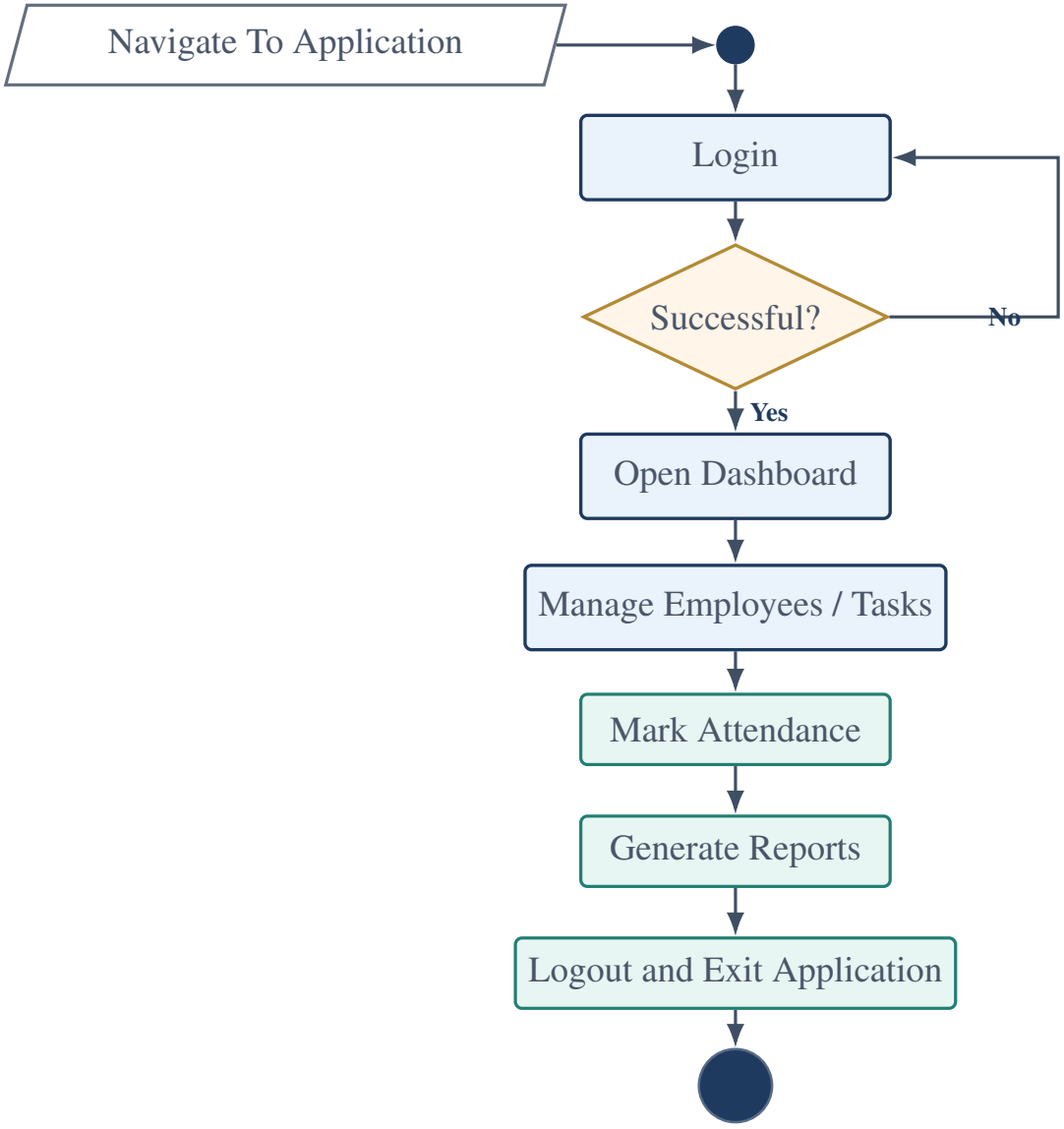

Figure 11.1: Activity Diagram

11.2 Use Case Diagram

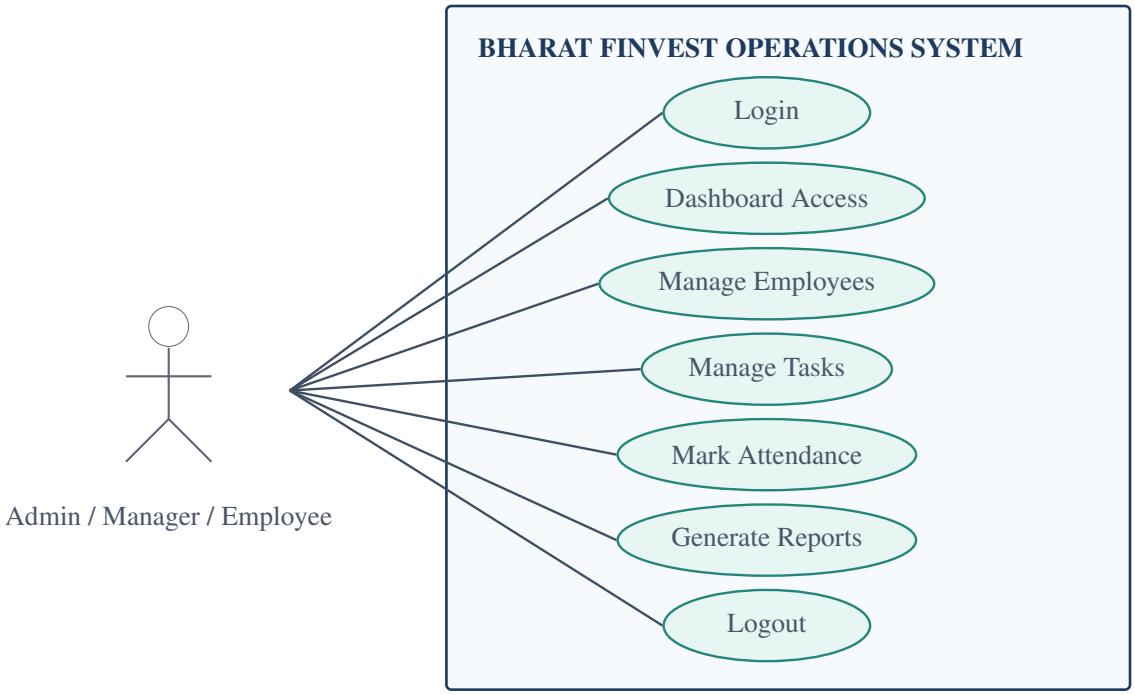

Figure 11.2: Use Case Diagram

11.3 Class Diagram

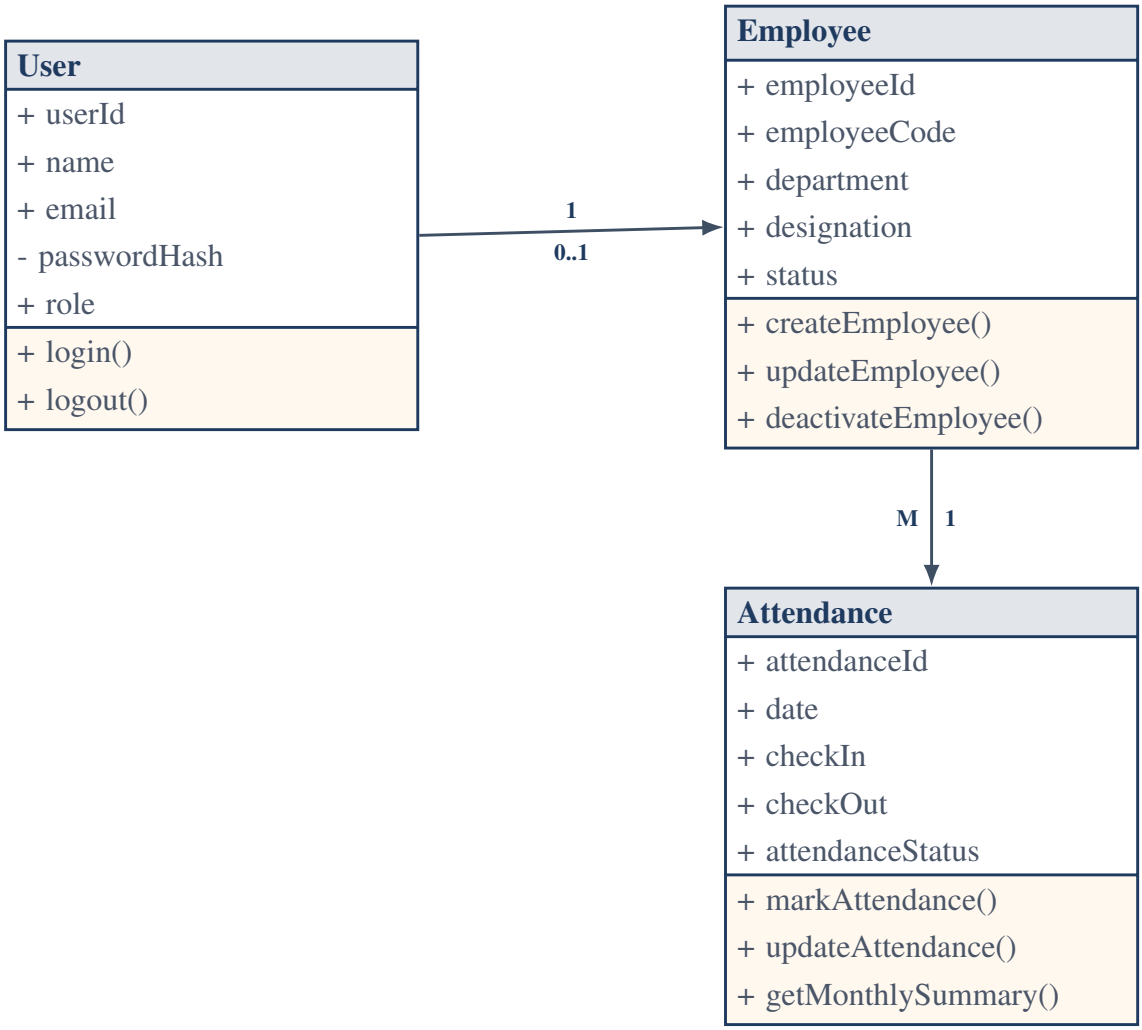

Figure 11.3: Class Diagram

11.4 Sequence Diagram

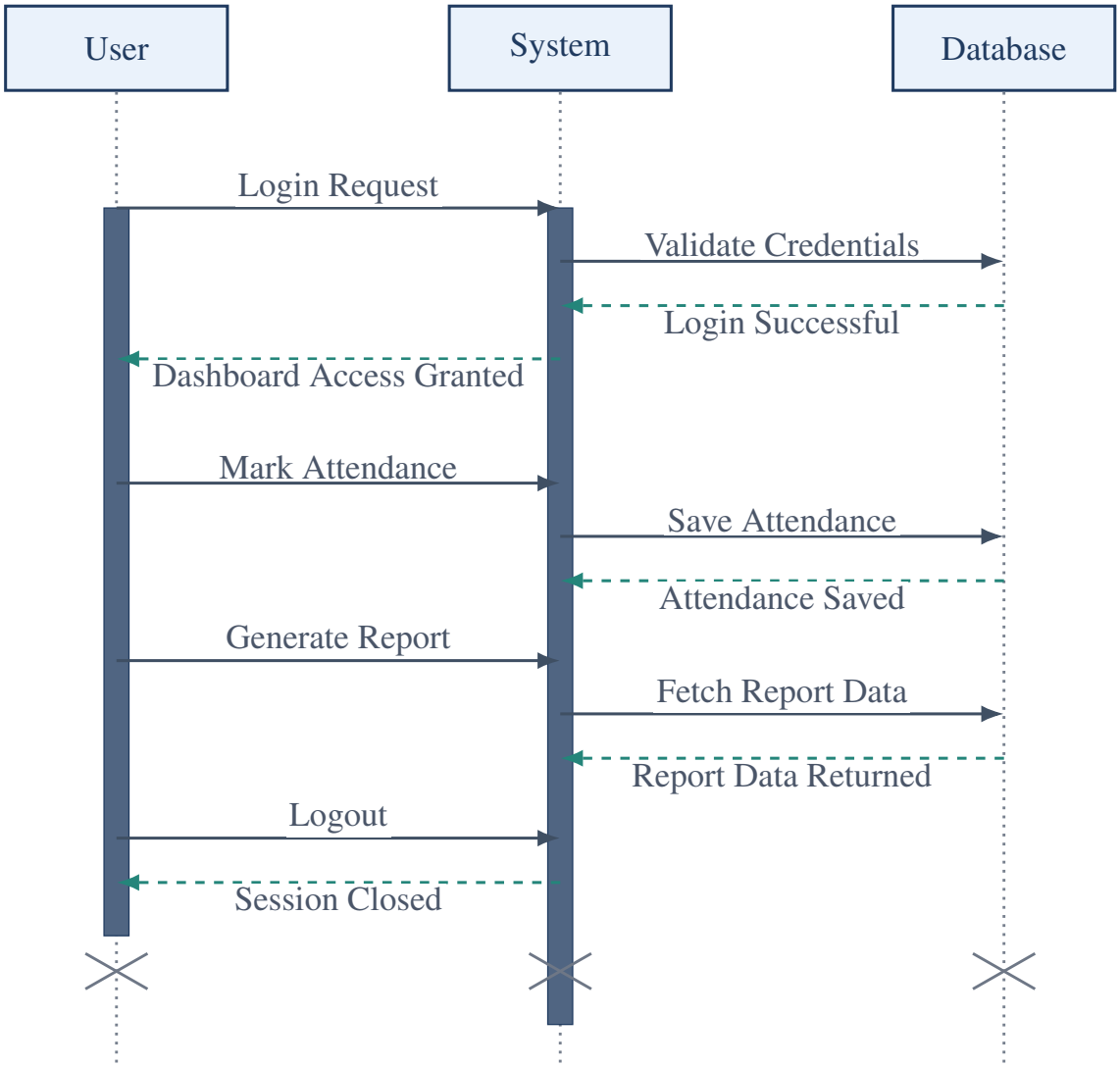

Figure 11.4: Sequence Diagram

MODULE DESCRIPTION

12.1 Login Module

What it does: Authenticates user credentials and initializes secure session state.

Frontend files:

- `frontend/src/pages/Login.jsx`
- `frontend/src/context/AuthContext.jsx`
- `frontend/src/api/axiosInstance.js`

Backend APIs involved:

- `POST /api/auth/login`
- `GET /api/auth/me`

Database tables: `users`

Role usage: Admin, Manager, and Employee.

12.2 Admin Dashboard Module

What it does: Displays operational KPIs and charts such as employee count, task status distribution, and today's attendance summary.

Frontend files:

- `frontend/src/pages/Dashboard.jsx`
- `frontend/src/components/StatCard.jsx`

Backend API involved: `GET /api/dashboard/stats`

Database tables: `employees`, `tasks`, `attendance`

12.3 Employee Management Module

What it does: Provides employee search, pagination, add, edit, and activate/deactivate operations.

Frontend files:

- `frontend/src/pages/Employees.jsx`
- `frontend/src/components/Avatar.jsx`

Backend APIs involved:

- `GET /api/employees`
- `GET /api/employees/:id`
- `POST /api/employees`
- `PUT /api/employees/:id`
- `DELETE /api/employees/:id`

Database tables: `employees`, optionally linked to `users` through `user_id`

Role behavior:

- Admin and Manager can add/edit employees.
- Only Admin can deactivate/reactivate employees.

12.4 Task Management Module

What it does: Supports task creation, update, filtering by status/priority, and deletion.

Frontend files:

- `frontend/src/pages/Tasks.jsx`

Backend APIs involved:

- `GET /api/tasks`
- `GET /api/tasks/stats`
- `GET /api/tasks/:id`
- `POST /api/tasks`

- `PUT /api/tasks/:id`
- `DELETE /api/tasks/:id`

Database tables: `tasks`, `employees`, `users`

12.5 Attendance Management Module

What it does: Tracks daily attendance with filtering and monthly summary.

Frontend files:

- `frontend/src/pages/Attendance.jsx`

Backend APIs involved:

- `GET /api/attendance`
- `POST /api/attendance`
- `PUT /api/attendance/:id`
- `GET /api/attendance/summary`

Database tables: `attendance`, `employees`

12.6 Reports Module

What it does: Generates PDF reports for employees, tasks, and attendance summary using client-side export.

Frontend files:

- `frontend/src/pages/Reports.jsx`

Backend APIs involved:

- `GET /api/employees`
- `GET /api/tasks`
- `GET /api/attendance/summary`

Libraries: `jspdf`, `jspdf-autotable`

12.7 Profile, Branding, and Avatar Module

What it does: Provides consistent visual identity and dynamic initials-based avatars.

Frontend files:

- `frontend/src/components/Avatar.jsx`
- `frontend/src/components/BrandLogo.jsx`
- `frontend/src/components/Sidebar.jsx`

Highlights:

- Removes dependency on static default avatar image for all users.
- Generates initials from real names (for example, RB, NY, PS, KD).
- Uses text-based safe branding when official logo asset is unavailable.

12.8 DevOps Module

What it does: Standardizes run, verification, health monitoring, CI, and data safety operations.

Files:

- `docker-compose.yml`
- `backend/Dockerfile`
- `frontend/Dockerfile`
- `frontend/nginx.conf`
- `.github/workflows/ci.yml`
- `scripts/backup-db.bat`
- `scripts/restore-db.bat`
- `start-project.bat`

API: `GET /api/health`

USER INTERFACE SCREENS

This section documents the primary user interface screens captured from the implemented Bharat Finvest Operations Portal.

13.1 Login Screen

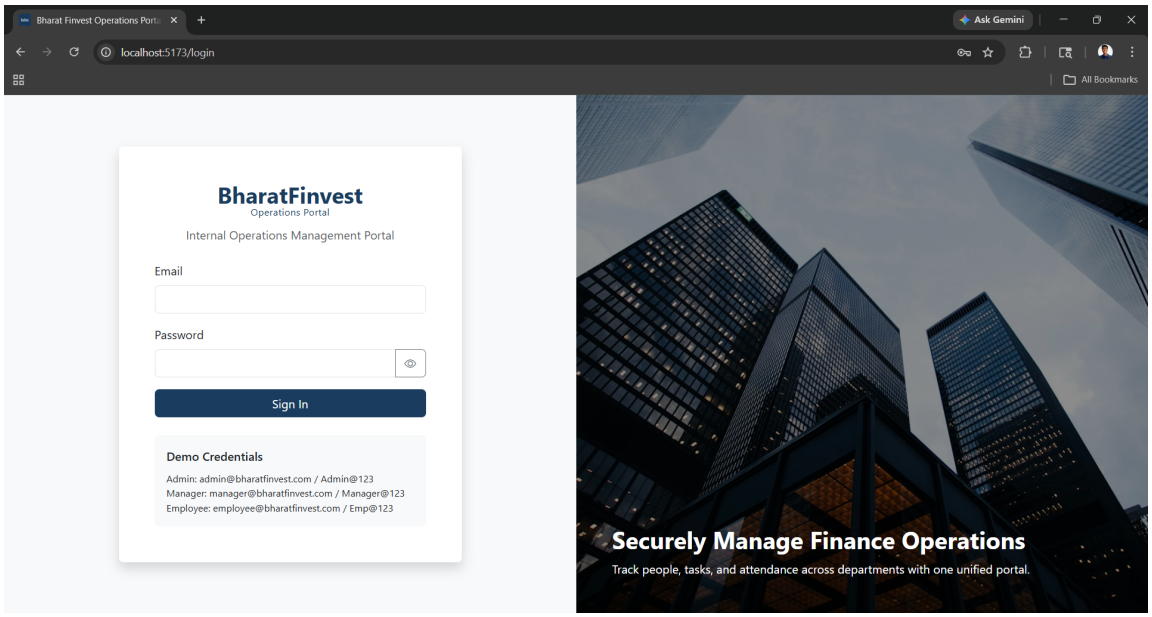

Figure 13.1: Login Screen

13.2 Dashboard Screen

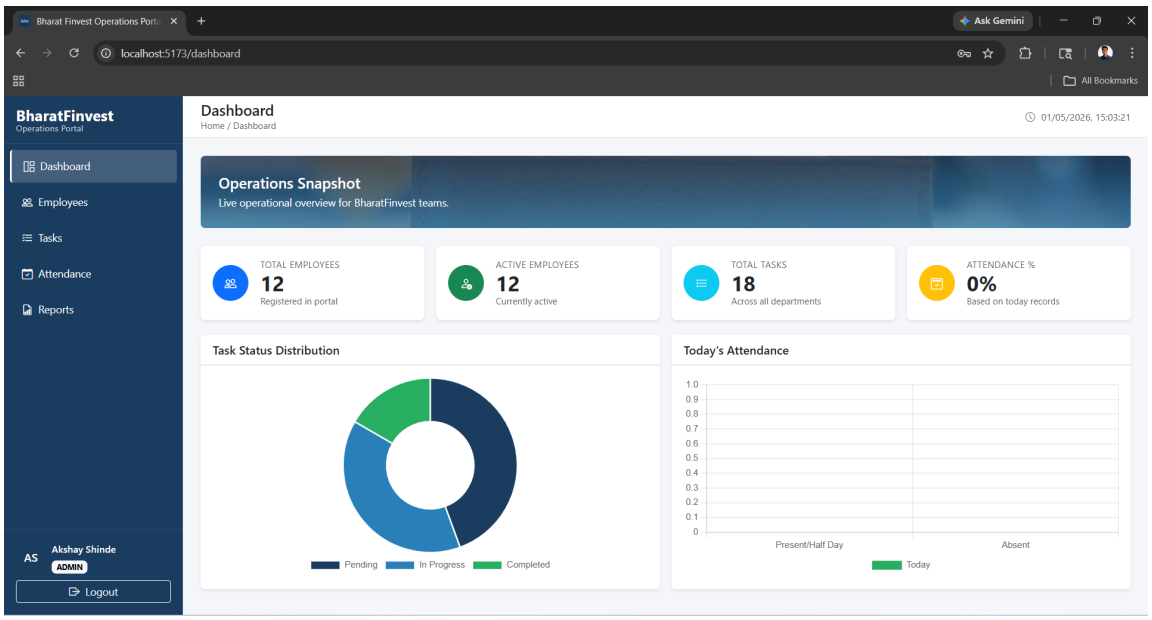

Figure 13.2: Dashboard Screen

13.3 Employees Screen

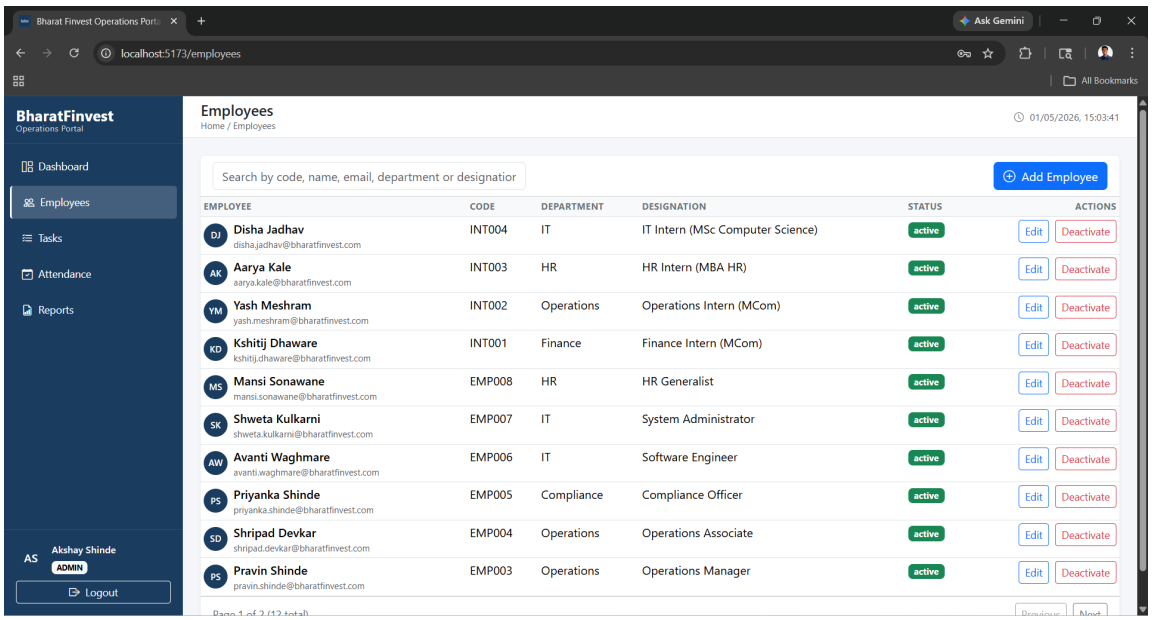

Figure 13.3: Employees Screen

13.4 Tasks Screen

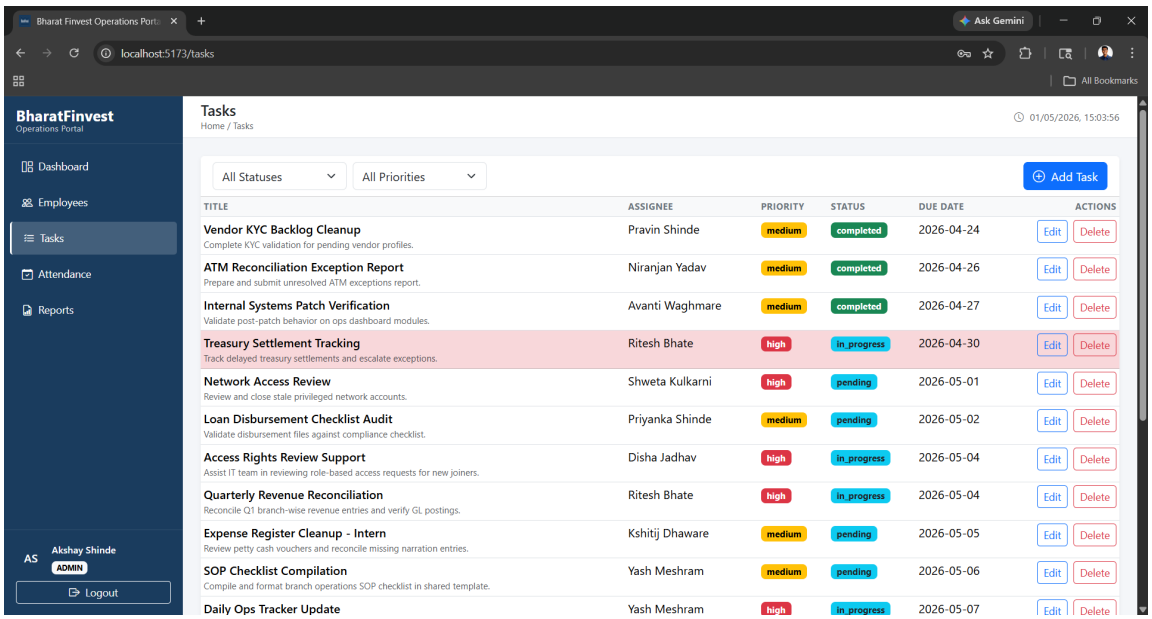

Figure 13.4: Tasks Screen

13.5 Attendance Screen

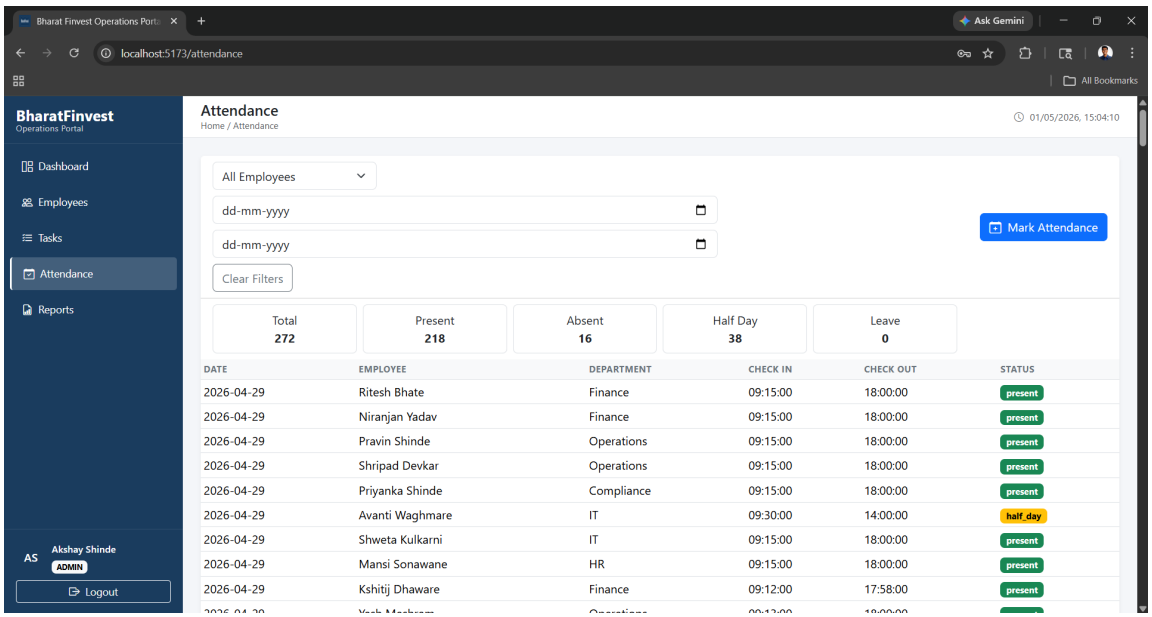

Figure 13.5: Attendance Screen

13.6 Reports Screen

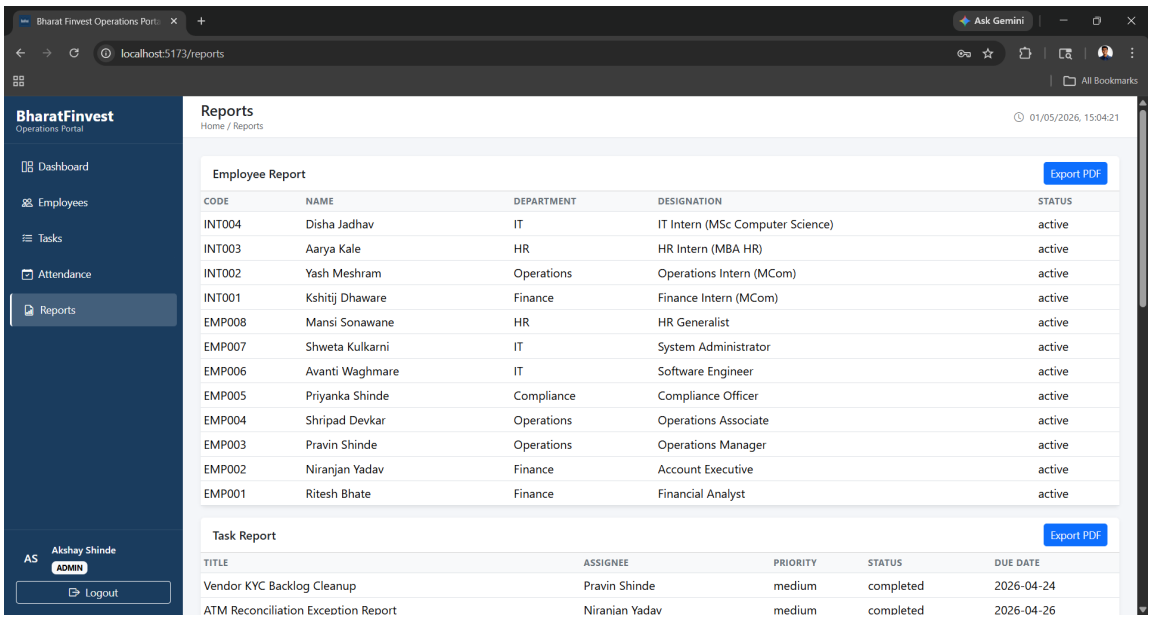

Figure 13.6: Reports Screen

API DESIGN

All APIs are exposed by the backend under the `/api` prefix and return JSON responses.

14.1 Authentication APIs

Method	Endpoint	Access	Purpose
POST	`/api/auth/login`	Public	Validates email and password. On success returns `token` and `user`.
GET	`/api/auth/me`	Authenticated	Returns latest user profile from database for current JWT.

Login Request Example

```
{
  "email": "admin@bharatfinvest.com",
  "password": "Admin@123"
}
```

Login Response Example

```
{
  "token": "<jwt_token>",
  "user": {
    "id": 1,
    "name": "Akshay Shinde",
    "email": "admin@bharatfinvest.com",
    "role": "admin",
    "department": "Administration"
  }
}
```

14.2 Dashboard API

Method	Endpoint	Access	Purpose
GET	`/api/dashboard/stats`	Authenticated	Returns employee counts, task stats, and today's attendance summary for dashboard widgets and charts.

14.3 Employee APIs

Method	Endpoint	Access	Purpose
GET	`/api/employees`	Authenticated	Paginated and searchable list of employees (supports page, limit, search).
GET	`/api/employees/:id`	Authenticated	Employee detail with task count and attendance percentage.
POST	`/api/employees`	Admin, Manager	Creates employee record.
PUT	`/api/employees/:id`	Admin, Manager	Updates employee profile data or status.
DELETE	`/api/employees/:id`	Admin	Soft deactivates employee (status set to inactive).

14.4 Task APIs

Method	Endpoint	Access	Purpose
GET	`/api/tasks`	Authenticated	Returns task list with optional filters: status, priority, assigned_to.
GET	`/api/tasks/stats`	Authenticated	Returns aggregate task status counts.
GET	`/api/tasks/:id`	Authenticated	Returns one task with assignee and assigner names.
POST	`/api/tasks`	Admin, Manager	Creates a new task linked to employee and assigner user.
PUT	`/api/tasks/:id`	Admin, Manager	Updates task fields.
DELETE	`/api/tasks/:id`	Admin	Permanently deletes task record.

14.5 Attendance APIs

Method	Endpoint	Access	Purpose
GET	`/api/attendance`	Authenticated	Returns attendance records with filters: employee_id, from, and to.
POST	`/api/attendance`	Admin, Manager	Marks attendance for employee/date; duplicate entries are handled with HTTP 409 conflict response.
PUT	`/api/attendance/:id`	Admin, Manager	Updates an existing attendance row.
GET	`/api/attendance/summary`	Authenticated	Returns monthly summary per employee with attendance percentage.

14.6 Health API

Method	Endpoint	Access	Purpose
GET	`/api/health`	Public	Performs `SELECT 1`; reports application and database health for monitoring.

14.7 API Error Strategy

- Validation failures return HTTP 400 with JSON message.
- Unauthorized and forbidden access return HTTP 401 and 403 respectively.
- Duplicate key or foreign-key conflicts return specific status codes (409 or 422).
- Unknown routes return HTTP 404 JSON via fallback middleware.
- Unhandled server errors are standardized through global error handler.

SECURITY IMPLEMENTATION

15.1 Authentication Security

- User authentication uses JSON Web Token (JWT).
- Login credentials are validated against hashed passwords in database.
- Access token is required in `Authorization: Bearer <token>` header for protected APIs.
- Backend middleware verifies token signature and expiration.

15.2 Password Security with bcrypt

- Passwords are not stored in plaintext.
- `bcrypt.compare()` is used during login to verify entered password against stored hash.
- Seeded demo accounts include pre-generated bcrypt hashes.

15.3 Role-Based Access Control

- `authMiddleware` authenticates request and populates `req.user`.
- `allowRoles(...)` middleware restricts access by role.
- Examples:
  - Employee deactivation and task deletion are admin-only.
  - Create/update employee/task/attendance are admin/manager-level actions.

15.4 Database and Query Security

- Backend uses `mysql2/promise` with parameterized queries.
- Query placeholders prevent SQL injection on dynamic inputs.

- Constraints such as unique keys and foreign keys protect data integrity.

15.5 Client-Side Session Handling

- Token and user data are stored in localStorage for session continuity.
- On app load, `/api/auth/me` refreshes user profile from backend.
- Axios interceptor handles 401 responses by clearing local state and redirecting to login.

15.6 Operational Security Hygiene

- `.env` is excluded from version control.
- `node_modules` and SQL backup dumps are ignored in `.gitignore`.
- CORS origin is explicitly set to `http://localhost:5173` in backend app configuration.

DEVOPS IMPLEMENTATION

16.1 Dockerization Strategy

The project is container-ready with separate services for MySQL, backend API, and frontend web server.

- `backend/Dockerfile`: builds Node.js backend image and runs `server.js` on port 5000.
- `frontend/Dockerfile`: builds React app and serves static output with Nginx.
- `frontend/nginx.conf`: handles SPA fallback routing and proxies `/api` requests to backend service.

16.2 Docker Compose Orchestration

`docker-compose.yml` defines three services:

- **mysql**: MySQL 8 container with initialization from `database/` scripts.
- **backend**: Depends on healthy MySQL and exposes port 5000.
- **frontend**: Depends on healthy backend and exposes port 80.

16.3 Health Checks

- MySQL health check uses `mysqladmin ping`.
- Backend health check calls `GET /api/health` from inside container.
- Health endpoint runs `SELECT 1` query to confirm database connectivity.
- If database check fails, endpoint returns HTTP 503 with `database: disconnected`.

16.4 GitHub Actions CI Pipeline

`.github/workflows/ci.yml` automatically runs on push and pull request.

- Sets up Node.js 20.
- Installs backend dependencies using `npm install`.
- Verifies backend app import with `node -e "require('./backend/src/app')"`-style check.
- Installs frontend dependencies and executes `npm run build`.

This pipeline quickly catches import errors and frontend build failures before deployment.

16.5 Windows Automation Scripts

1. One-click startup script

`start-project.bat`:

- Detects Node/npm availability.
- Installs backend/frontend dependencies only if `node_modules` is missing.
- Starts backend and frontend in separate CMD windows.
- Opens browser at `http://localhost:5173`.

2. Backup script

`scripts/backup-db.bat`:

- Uses full-path `mysqldump.exe`.
- Creates `backups/` if missing.
- Produces timestamped SQL dump file for `bharat_finvest_ops`.

3. Restore script

`scripts/restore-db.bat`:

- Accepts drag/drop backup file path from user.
- Uses full-path `mysql.exe` to restore selected dump into target database.

16.6 Environment-based Configuration

Backend configuration uses environment variables from `backend/.env`: `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`, `JWT_EXPIRES_IN`, and `PORT`.

This design supports local execution, Docker execution, and future production deployment with minimal changes.

TESTING STRATEGIES

The project validation was performed using a layered testing strategy to improve reliability and demo safety.

17.1 Unit-level Validation

Full formal unit test suites are not yet introduced with Jest/Mocha; however, core logic was validated at function level through controlled API requests and deterministic input scenarios.

Examples:

- Payload validation for employee/task/attendance controllers.
- Duplicate handling paths (attendance duplicate entry, employee duplicate keys).
- JWT middleware accept/reject flows.

17.2 Integration Testing

Integration testing focused on API-to-database behavior.

- CRUD flows were tested against real MySQL schema.
- Foreign key constraints were verified through valid/invalid references.
- Dashboard and summary APIs were validated for aggregated SQL output.

17.3 System Testing

System testing validated end-to-end behavior from browser to database.

- Login and protected routing.
- Employee, task, and attendance operation sequence.
- PDF generation in reports page.
- Logout and stale session fallback behavior.

17.4 UI/UX Testing

UI testing was executed with focus on visible controls and modal flows.

- All actionable buttons were verified for expected behavior.
- Required fields and form validation states were checked.
- Success/error banners were verified after API responses.
- Loading states and disabled buttons were verified during requests.

17.5 Build and Runtime Verification

- Backend import verification: `node -e "require('./src/app')"`
- Frontend production build verification: `npm run build`
- Health endpoint verification: `GET /api/health`
- CI build verification in GitHub Actions pipeline.

17.6 Regression Safety

Changes for branding, avatar logic, and DevOps enhancements were validated to ensure no break in existing core modules:

- Login module remains functional.
- Dashboard metrics continue loading.
- CRUD operations remain stable.
- PDF exports continue generating non-empty documents.

TEST CASES

ID	Test Case	Input / Action	Expected Result	Status
TC01	Login success	Valid admin credentials	Redirect to dashboard with active session and sidebar user details.	Pass
TC02	Login failure	Invalid password	Error alert shown for invalid credentials.	Pass
TC03	Auth refresh	Existing JWT, app reload	`/api/auth/me` refreshes user and updates localStorage state.	Pass
TC04	Employee list load	Open Employees page	Paginated data displayed with search and avatar initials.	Pass
TC05	Add employee	Submit valid Add Employee form	New employee created, success banner shown, table refreshed.	Pass
TC06	Edit employee	Update existing employee details	Employee updated, modal closes after success, table refreshed.	Pass
TC07	Deactivate employee	Admin clicks Deactivate and confirms	Employee status changes to inactive with success banner.	Pass
TC08	Task list filter	Apply status and priority filters	Task table shows matching filtered records only.	Pass
TC09	Add task	Submit valid task form with assignee	Task created and visible in refreshed list.	Pass
TC10	Edit task	Update task status or due date	Task updated while preserving relational links.	Pass
TC11	Delete task	Admin confirms task deletion	Task removed from database and UI list.	Pass
TC12	Mark attendance	Submit employee/date/status in modal	Attendance entry inserted and table refreshed.	Pass
TC13	Duplicate attendance guard	Mark same employee/date twice	API returns conflict; UI displays error in modal.	Pass
TC14	Attendance filters	Filter by employee and date range	Table updates with matching records.	Pass
TC15	Employee PDF export	Click Export PDF in Reports page	Downloaded PDF contains employee rows and header.	Pass
TC16	Task PDF export	Click Export PDF in Reports page	Downloaded PDF contains task data with timestamp.	Pass
TC17	Attendance PDF export	Click Export PDF in Reports page	Downloaded PDF contains attendance summary table.	Pass

ID	Test Case	Input / Action	Expected Result	Status
TC18	Logout flow	Click sidebar logout button	Token and user are removed; route redirects to login.	Pass
TC19	Health API success	GET `/api/health` with DB available	JSON response shows status OK and database connected.	Pass
TC20	Unauthorized route	Call protected API without token	HTTP 401 JSON error response returned.	Pass

Notes on Evidence Collection

- API responses were verified through browser-driven requests and backend logs.
- Frontend behavior was verified with modal actions, alerts, and table refresh checks.
- Build and import checks were also validated through command-line verification and CI workflow.

LIMITATIONS

Although the application is stable for academic demonstration and prototype usage, the following limitations are currently present:

- Role policy is practical for demo but can be expanded with finer permission granularity (module-level/field-level controls).
- Automated unit and integration test suites are not fully formalized with a dedicated testing framework.
- Audit trail for all data changes (who changed what and when) is not yet implemented as a separate module.
- Notification workflows (email/SMS reminders for overdue tasks or missing attendance) are not integrated.
- Large-scale pagination and server-side report batching are basic; advanced performance tuning can be introduced for high-volume production traffic.
- Deployment is currently local/container-ready; cloud production deployment pipeline is not yet configured.
- Mobile-first UX optimization can be enhanced for long operational tables on very small screens.

FUTURE SCOPE

The following enhancements are proposed for the next phase:

1. **Email Notifications:** Automated emails for task assignment, due reminders, and attendance irregularities.
2. **SMS Alerts:** Optional SMS notifications for urgent operational actions.
3. **Payroll Integration:** Connect attendance and role data with payroll calculations.
4. **Leave Management Module:** Add leave request, approval workflow, and leave balance tracking.
5. **Cloud Deployment:** Deploy backend/frontend/database to cloud infrastructure for wider usage.
6. **Jenkins Pipeline:** Add an enterprise CI/CD option alongside GitHub Actions.
7. **Advanced Analytics:** Department productivity trends, SLA tracking, and predictive summaries.
8. **Audit Logs:** Immutable activity log for compliance and security review.
9. **Document Management:** Upload and attach operational documents to tasks or employee profiles.
10. **Role Customization:** Configurable permissions with policy templates per business unit.

These additions would transition the current prototype into a more comprehensive enterprise operations platform.

CONCLUSION

The **Bharat Finvest Internal Operations Management System** was successfully designed and implemented as a full-stack industrial training project for M.Sc. (Computer Science) Part II, Sem IV.

This project was developed during industrial training at **Nityashree Infosystems Pvt. Ltd.** as a client-style internal operations management system for **Bharat Finvest**.

The final system demonstrates:

- Secure login with JWT and role-aware authorization.
- Practical employee, task, and attendance workflow management.
- Dashboard-level operational visibility through computed metrics and charts.
- Exportable PDF reports for documentation and review needs.
- Production-oriented operational support through Docker, health checks, CI, and backup/restore scripts.

The project aligns with both academic objectives and real-world operational requirements by combining software engineering principles, relational database design, API development, and DevOps practices in one coherent solution.

Hence, the project is suitable for final presentation, academic assessment, and further extension toward production readiness. This report does not claim official production deployment within Bharat Finvest.

BIBLIOGRAPHY & REFERENCES

1. Bharat Finvest Ops project repository: https://github.com/tejasskamble/bharat-finvest-ops-system
2. Project source files inspected directly from local repository (backend, frontend, database, and DevOps files).
3. Reference format document: MSC IT Project report Cover Page(1).docx
4. Certificate format reference: MSC IT certificates.pdf
5. Sample report structure reference: Final_It_Report.pdf
6. Internal evaluation guideline reference: IT Project Internal Evalution guideline.pdf
7. React documentation: https://react.dev/
8. Vite documentation: https://vitejs.dev/
9. Express documentation: https://expressjs.com/
10. MySQL 8 documentation: https://dev.mysql.com/doc/
11. JWT introduction: https://jwt.io/introduction
12. bcrypt package documentation: https://www.npmjs.com/package/bcrypt
13. Docker documentation: https://docs.docker.com/
14. GitHub Actions documentation: https://docs.github.com/actions
15. jsPDF documentation: https://github.com/parallax/jsPDF

APPENDIX

23.1 A. Training and Client Context

- Training Organization: **Nityashree Infosystems Pvt. Ltd.**
- Client / Project Domain Organization: **Bharat Finvest**
- Project Title: **Bharat Finvest Internal Operations Management System**

23.2 B. GitHub Repository

`https://github.com/tejasskamble/bharat-finvest-ops-system`

23.3 C. Demo Credentials

Role	Email	Password
Admin	`admin@bharatfinvest.com`	`Admin@123`
Manager	`manager@bharatfinvest.com`	`Manager@123`
Employee	`employee@bharatfinvest.com`	`Emp@123`

23.4 D. Employee and Intern Seed Dataset

- Ritesh Bhate
- Niranjan Yadav
- Pravin Shinde
- Shripad Devkar
- Priyanka Shinde
- Avanti Waghmare
- Shweta Kulkarni

- Mansi Sonawane
- Kshitij Dhaware
- Yash Meshram
- Aarya Kale
- Disha Jadhav

23.5 E. Essential Commands

1. Database Import (Windows CMD)

```
mysql -u root -p < database\schema.sql
mysql -u root -p bharat_finvest_ops < database\seed.sql
```

Note: If mysql is not in PATH, use the full executable path of mysql.exe.

2. Backend Run

```
cd backend
npm install
npm run dev
```

3. Frontend Run

```
cd frontend
npm install
npm run dev
```

4. One-Click Run

```
start-project.bat
```

5. Health Check

```
http://localhost:5000/api/health
```

6. Docker Commands

```
docker-compose up --build
docker-compose down
```

7. Backup and Restore

```
scripts\backup-db.bat
scripts\restore-db.bat
```

23.6 F. Final QA Checklist Reference

- Login tested
- Dashboard tested
- Employee CRUD tested
- Task CRUD tested
- Attendance tested
- Reports PDF tested
- Logout tested
- Health API tested